



ASUS joins WP8 brigade?

According to reports, ASUSTek is set to jump in the Windows Phone 8 market with a WP8 version of its popular PadFone

Developers rejoice!

Google recently added support for ARM in Chrome's Native Client making the job of developers easy

DIY

Giving some structure to our blog

Now let's start actually building our CMS. We will start with our models file that defines the data structure of our app.

We will have three kinds of objects in our database, the first is a Person object. This is just a first and last name. Second, we have a Role object, for the role a person plays in a movie; just a name is enough here. Finally we have a Movie, which has a title, and an arbitrary number of people, with different roles.

```
class Person(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    def __unicode__(self):
        return "%s %s" % (self.first_name, self.last_name)
```

The Person class is subclassing the Model class provided by Django, which ensures that we get all the benefits of Django automatically. This will automatically create a database table and provide a rich API. We have kept the person model simple, just the name, but we could add a whole lot more. Each of these becomes a column in the database table, in this case a character field with a maximum length of 50 characters.

The `__unicode__` method is meant to return a textual representation of this object; here that is a combination of the first and last name.

What we need next is a Role class to define the possible roles a person can have. Rather than hardcoding it, we leave it flexible, so that more roles can be added in the future.

```
class Role(models.Model):
    name = models.CharField(max_length=50)
    def __unicode__(self):
        return self.name
```

This is pretty straightforward compared to what we already did, if you noticed.

Finally, we need a `@Movie@` class for each movie; we will keep this simple as well:

```
class Movie(models.Model):
    title = models.CharField(max_length=256)
    summary = models.TextField()
    release_date = models.DateField()
    def __unicode__(self):
        return self.title
```

Here we have two new fields, a `TextField` which allows for a greater bulk of text, and a `DateField` for the release date.

Although now we have all the basic data, we need some way to connect them, a way to say which Person worked on what Movie

in what Role. We'll call this data model `Credit`; here is how we define that:

```
class Credit(models.Model):
    movie = models.ForeignKey(Movie)
    role = models.ForeignKey(Role)
    person = models.ForeignKey(Person)
```

`ForeignKey` is used to link one kind of data to another. So a blog post can have a `ForeignKey` to an author, or a product object can have one for a manufacturer; it is necessary to create some kind of hierarchy.

With this we have defined the structure of the data our app will work with, but we still need to tell it how everything works together.

Before we do that though, let's have Django create the basic structure for our database; run the following command in the main project folder:

```
python manage.py syncdb
```

This creates database tables for our models, and allow you to set up an admin username and password; keep note of that.

The admin panel

The great thing about Django is that even with this, our site already has an admin panel, it just doesn't know about our new data yet. So we need to tell it. Create a new file called `admin.py` in the `movies` directory, and add the following code to it:

```
from movies.models import Credit, Movie, Person, Role
from django.contrib import admin
admin.site.register(Credit)
admin.site.register(Movie)
admin.site.register(Person)
admin.site.register(Role)
```

Now, run the following command in the project directory:

```
python manage.py runserver
```

This will launch the server for our website, by default at `http://127.0.0.1:8000`. The real magic right now is at `http://127.0.0.1:8000/admin` which hosts the admin panel. Enter the password you used while running the `syncdb` command, and you will see what we have built.

You will see that there is already quite a functional admin system! You can add movies, people, roles and credits. But, the way we add credits isn't very convenient, wouldn't it be better to highlight the credits for the movie under it?

The good thing is that the admin panel is not only automatically generated, but also highly customisable.

Just like we defined a custom data structure for our app using models, we can define a custom admin panel using `admin` models. We want to add credits from within a movie, for this

ASRock
Get One, The Best One

ASRock Motherboards awarded by DIGIT & CHIP

Model	Image
Z77 Extreme6	
X79 Extreme6	
Z68 Extreme7 Gen3	
Z68 Extreme3 Gen3	
A75 Extreme6	
A55M-HVS	
Fatal1ty 990FX Professional	